Tesla Stock Price Prediction

Using Multimodal Deep Learning

Detailed Project Report

UTS 49275 Neural Networks and Fuzzy Logic

Project focus: returns-based forecasting with technical indicators, market context, sentiment features, and sequence models.

Generated: 2026-04-27 15:57

Executive Summary

Project Aim

This project builds a multimodal forecasting system for Tesla stock. The goal is to predict future stock movement by combining numerical market history, engineered technical indicators, broader market context, and sentiment-style signals.

Instead of predicting raw closing prices directly, the project predicts percentage returns and reconstructs prices afterward. This is a stronger modelling choice because returns are usually more stationary than price levels.

Main Contributions

- End-to-end pipeline covering data collection, feature engineering, preprocessing, model training, inference, and dashboard visualisation.
- Leakage-aware preprocessing that fits scalers on the training portion only.
- Multimodal fusion model with separate price and sentiment encoders.
- Baseline comparison with LSTM, GRU, Transformer, and XGBoost implementations.
- Interactive Streamlit dashboard for forecasts, charts, model comparison, and explainability.

Problem Definition and Objectives

Forecasting Task

The primary task is next-day return regression. The model predicts the future percentage return, then reconstructs the next predicted close price using predicted_price = today_close * (1 + predicted_return).

The project also supports multi-horizon return prediction for 1, 3, 5, and 7 trading-day horizons. The secondary task is direction classification: returns are mapped into Down, Neutral, and Up classes using a symmetric neutral band of +/-0.5%.

Design Objectives

- Use chronological train, validation, and test splits to respect time ordering.
- Avoid data leakage by fitting feature and target scalers only on the training split.
- Compare the main multimodal fusion model against simpler standalone baselines.
- Keep the pipeline reproducible by caching raw data and using synthetic sentiment by default for long historical runs.

Data Sources and Feature Engineering

Market Data

The stock-data layer retrieves Tesla OHLCV data from Yahoo Finance and stores raw data under data/raw/. Base features include open, high, low, close, volume, daily returns, log returns, price range, and price range percentage.

The feature-engineering layer adds market context using SPY and VIX. The implementation supports cached SPY/VIX files and can disable market context through USE_MARKET_CONTEXT when offline reproducibility is preferred.

Technical Indicators

- Trend indicators: SMA, EMA, MACD, and MACD signal/histogram.
- Volatility indicators: Bollinger Bands, ATR, rolling volatility, and Parkinson volatility.
- Volume and price-position indicators: OBV, VWAP, stochastic oscillator, momentum, rate of change, candlestick body, shadows, and gap features.
- Calendar features: day of week, month sine/cosine, month end, and quarter end.

Sentiment Data

The project supports real RSS headline sentiment and synthetic sentiment. RSS mode fetches currently available headlines and scores them with VADER. Synthetic mode creates sentiment-like features from lagged returns, lagged rolling trends, volatility, and noise.

For long historical experiments, synthetic sentiment is the more reproducible default. Live RSS mode does not provide true historical headline coverage, so older dates are mostly neutral-filled unless a dated historical news source is added.

Preprocessing Methodology

Data Preparation

DataPreprocessor merges stock, technical, market, calendar, and sentiment data by date. Missing values and infinities are cleaned using forward/backward filling followed by removal of remaining invalid rows.

Feature columns and sentiment columns are separated so the multimodal model can receive different tensors for the price/technical stream and the sentiment stream.

Leakage Control

The most important methodological safeguard is fitting scalers only on the training portion of the data. This prevents validation/test statistics from influencing the learned transformations.

After scaler fitting, all rows are transformed using train-fitted scalers. Sliding windows are then created with a configured 60-day sequence length.

Targets

The legacy single-step target is Target_Return. Multi-horizon targets are Target_Return_1d, Target_Return_3d, Target_Return_5d, and Target_Return_7d. Direction labels are derived from the 1-day return using the configured neutral threshold.

Pipeline Architecture

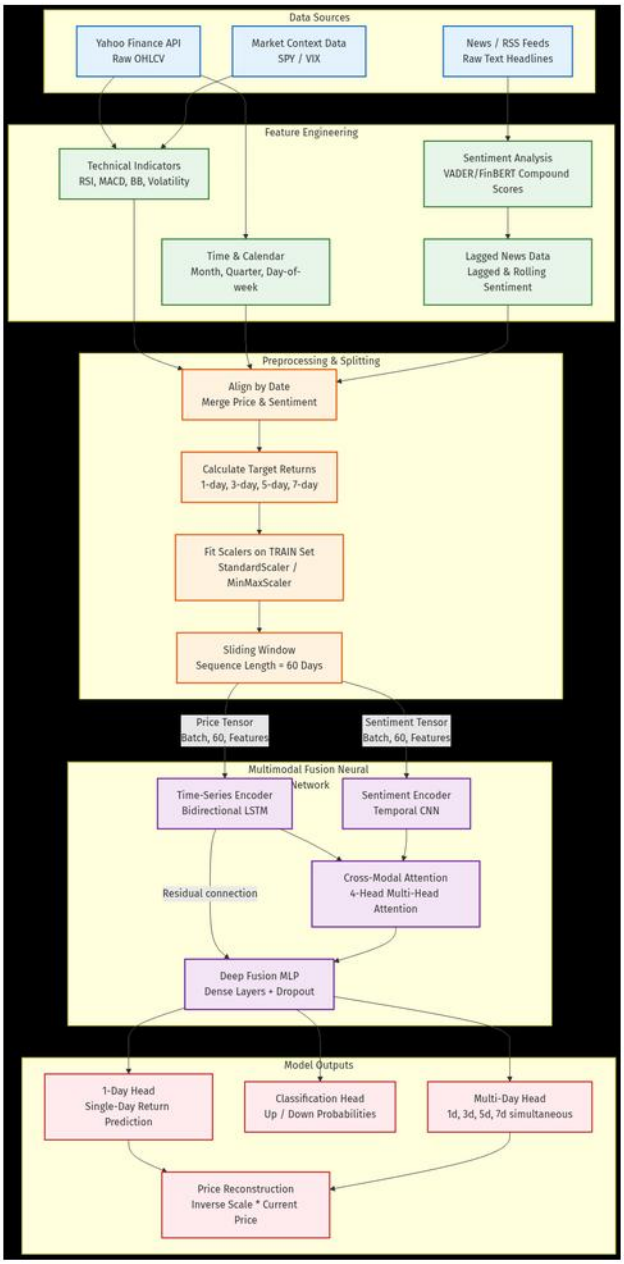

Figure 1. Project pipeline from data collection through feature engineering, preprocessing, sequence modelling, and output heads.

Model Architecture

Multimodal Fusion Model

The main model uses two aligned temporal streams. The price stream contains OHLCV, technical, market, and calendar features. The sentiment stream contains daily sentiment scores and lagged sentiment fields.

The price branch uses an LSTM-based time-series encoder with attention. The sentiment branch uses a temporal sentiment encoder. Their representations are fused through cross-modal attention and a feed-forward fusion network.

Output Heads

- Single-day regression head predicts the next-day scaled return.
- Multi-day regression head predicts returns for the configured 1, 3, 5, and 7 day horizons.
- Classification head predicts Down, Neutral, or Up movement.

Baseline Models

model_comparison.py trains standalone LSTM, GRU, Transformer, and XGBoost regressors using the same preprocessing pipeline. These baselines help determine whether the multimodal fusion design improves over simpler sequence or tabular approaches.

Training and Evaluation

Training Setup

The fusion model is trained with AdamW, OneCycleLR scheduling, gradient clipping, and fixed-epoch training. The combined loss uses Smooth L1 regression loss, multi-horizon Smooth L1 loss, and cross-entropy direction-classification loss.

The split proportions are configured as 80% training, 10% validation, and 10% test, applied chronologically.

Metrics

- RMSE: root mean squared dollar error after reconstructing predicted price.
- MAE: mean absolute dollar error after reconstructing predicted price.
- MAPE: percentage price error.
- Direction Accuracy: percentage of correct Down/Neutral/Up classifications.

Experimental Results

Saved Baseline Results

- LSTM: RMSE $13.51, MAE $10.51, MAPE 3.08%, Direction Accuracy 49.9%
- GRU: RMSE $14.56, MAE $11.06, MAPE 3.25%, Direction Accuracy 49.1%
- XGBoost: RMSE $14.71, MAE $11.51, MAPE 3.31%, Direction Accuracy 49.1%

Interpretation

The saved comparison artifact shows LSTM with the lowest RMSE among the currently saved baseline runs. GRU and XGBoost are close behind, suggesting that recurrent sequence models and flattened tree-based baselines both capture useful signal from the engineered feature set.

The current codebase includes a Transformer baseline, but the checked-in comparison CSV does not include a Transformer row. For final submission consistency, rerun model_comparison.py when time and dependencies permit, or explicitly state that the saved artifact reflects the completed LSTM, GRU, and XGBoost runs.

Training History

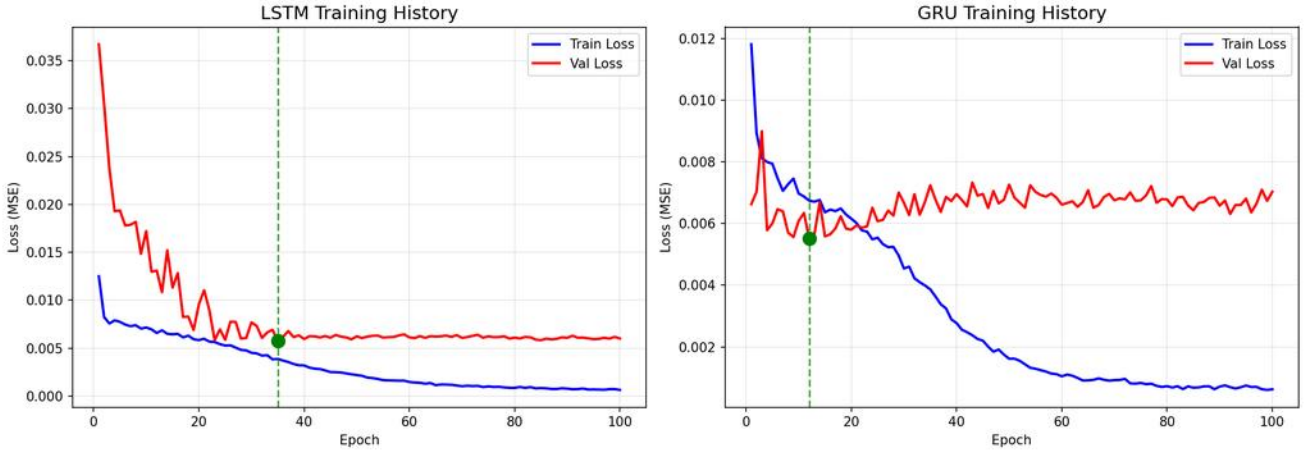

Figure 2. Saved training history visualisation for the project.

Baseline Comparison Chart

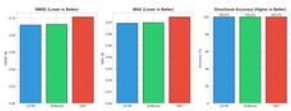

Figure 3. Saved baseline model comparison chart.

Dashboard and Inference

CLI Inference

predict.py loads the trained fusion checkpoint and preprocessing metadata from models/. It fetches recent market data, computes indicators, transforms features with the saved scaler, extracts the latest 60-day window, and produces price and direction forecasts.

Streamlit Dashboard

The dashboard provides data visualisation, technical indicator plots, prediction cards, model comparison tables, live predictions from saved baselines, and SHAP-based feature analysis through an XGBoost surrogate.

The dashboard uses synthetic sentiment by default so it remains fast and less dependent on RSS/network availability.

Limitations and Risk Analysis

Methodological Limitations

- Stock prices are noisy, non-stationary, and strongly affected by events not present in historical OHLCV data.
- Live RSS sentiment is not true historical sentiment. Without a historical news database, older dates in long training windows are mostly neutral-filled.
- One fixed chronological split is useful but weaker than walk-forward validation for robust time-series evaluation.
- Direction accuracy around 50% indicates that directional prediction remains difficult and should not be interpreted as trading-grade performance.

Engineering Limitations

- Saved model artifacts must stay aligned with current code. The current code includes Transformer, while the saved comparison CSV does not yet include a Transformer result.
- Market context depends on Yahoo Finance data unless cached SPY/VIX files are already available.
- The project is educational and should not be used for real trading decisions.

Recommendations and Conclusion

Recommended Improvements

- Regenerate all baseline artifacts so LSTM, GRU, Transformer, and XGBoost results are present in the same comparison table.
- Add walk-forward validation to report performance stability across multiple market regimes.
- Add a dated historical news source for genuine historical sentiment rather than current RSS headlines.
- Add an experiment log that records date range, model configuration, sentiment mode, feature set, metrics, and artifact paths.
- Add smoke tests for preprocessing shapes, scaler loading, model forward-pass output keys, and CLI inference startup.

Conclusion

The project demonstrates a complete multimodal stock forecasting workflow with thoughtful modelling choices: return-based targets, chronological splitting, train-only scaling, multi-horizon outputs, and baseline comparison. Its main value is as a rigorous educational pipeline for financial sequence modelling rather than as a production trading system.

References

Core References

- Hochreiter, S. and Schmidhuber, J. Long Short-Term Memory. Neural Computation, 1997.
- Cho, K. et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, 2014.
- Chen, T. and Guestrin, C. XGBoost: A Scalable Tree Boosting System, 2016.
- Araci, D. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models, 2019.
- Project source files: README.md, ARCHITECTURE.md, train.py, model_comparison.py, src/data, src/features, and src/models.